



SORBONNE UNIVERSITÉ
MASTER AI2D

Interface d'analyse et d'aide à la décision pour les jeux de fantasy cyclisme

PAI2D

Yuxiang ZHANG – Kenan ALSAFADI – Gildas DE MICHIEL

Encadrant : Thibaut LUST, LIP6

18 mai 2026

Table des matières

1	Introduction	2
2	État de l’art	2
2.1	Modélisation des jeux de fantasy cyclisme	3
2.2	Analyse de robustesse et de tolérance	3
2.3	Extensions multi-étapes du sac à dos	3
2.4	Positionnement de notre travail	3
3	Contributions	4
3.1	Présentation des règles du jeu	4
3.2	Modèle mathématique	4
3.2.1	Implémentation et résolution	6
3.3	Présentation de la solution optimale	7
3.3.1	Données utilisées	7
3.3.2	Équipe optimale	7
3.4	Analyse post-optimale	8
3.4.1	L’Analyse de sensibilité des points	8
3.4.2	Couches de Pareto	9
3.4.3	Stabilité par simulation Monte Carlo - Facteur Chance	10
3.4.4	Analyse d’un coureur forcé	11
4	Conclusion	11
A	Annexe A : Cahier des charges	13
B	Annexe B : Manuel utilisateur	15
B.1	Installation	15
B.2	Lancement de l’application	15
B.3	Interface principale	15
B.3.1	Barre latérale	15
B.3.2	Zone centrale – Optimisation classique	16
B.3.3	Zone centrale – Simulation Monte Carlo	16

1 Introduction

Les jeux de fantasy sport, sont des jeux où les participants interprètent le rôle de propriétaires d'équipes sportives. Ils défient d'autres joueurs et obtiennent des résultats basés sur des événements sportifs réels. En fonction des choix stratégiques effectués, le joueur de fantasy reçoit un certain nombre de points. Comme un véritable propriétaire d'équipe, le joueur peut vendre ou acheter les membres de son équipe. Le jeu "On connaît nos classiques, le jeu" (OCNC), organisé par la RTBF pour le Tour de France, en est une illustration. Chaque participant constitue une équipe virtuelle de coureurs professionnels, dans le but de cumuler le maximum de points au fil des étapes. Les choix initiaux sont contraints par une limite budgétaire et des règles de composition d'équipe. Les joueurs doivent également anticiper les aléas de la course : abandons et disponibilité de transferts pendant les journées de repos.

Le projet présenté dans ce rapport s'inscrit dans le cadre du Master AI2D (Algorithmes, Intelligence Artificielle, Interactions et Décision) de Sorbonne Université, sous la supervision de Thibaut Lust, enseignant-chercheur au LIP6. Nous avons deux objectifs à réaliser. D'une part, fournir une solution logicielle permettant de *calculer a posteriori* l'équipe optimale qui aurait remporté le jeu OCNC sur une édition donnée. D'autre part, exploiter cette solution pour mener une *analyse post-optimale approfondie* : identifier les coureurs indispensables, ou encore de mesurer la stabilité de la sélection optimale face à des perturbations. Une telle analyse peut être utile, par exemple pour un journaliste cherchant à expliquer les choix stratégiques qui apportent le plus de point.

Notre approche repose sur la programmation linéaire en nombres entiers (PLNE). Nous avons développé un modèle mathématique complet du jeu incluant les transferts à chaque période. Puis nous l'avons implémenté en Python à l'aide de la bibliothèque PuLP. Le code est structuré en modules distincts et accessible sur le dépôt Git suivant :

https://github.com/Kapiloten/PAI2D_Fantasy_Cyclisme

Ce dépôt contient l'ensemble des fichiers nécessaires à la reproduction des résultats. En effet, nous avons développé une interface interactive avec Streamlit. La documentation associée a été rédigée dans l'annexe B.

Le rapport est organisé comme ceci. La section 2 présente un état de l'art des méthodes d'optimisation pour les jeux de fantasy sport et les problèmes de sac à dos multi-étapes. La section 3 constitue les différentes contributions que nous avons réalisées lors de ce projet. Après un rappel du règlement d'OCNC (section 3.1), nous détaillons le modèle PLNE associé aux contraintes mathématiques (section 3.2). Ensuite, nous exposons les résultats obtenus par l'optimisation du PLNE (section 3.3). Puis, l'analyse post-optimale calculant la stabilité des joueurs (section 3.4). La section 4 résume les points forts et les faiblesses de notre travail et ouvre des perspectives. Enfin, les annexes rappellent le cahier des charges initial (Annexe A) et fournissent un manuel utilisateur (Annexe B).

2 État de l'art

L'optimisation *ex-post* des jeux de fantasy sport s'appuie sur une littérature mélangeant à la fois la recherche opérationnelle et l'analyse des données sportives. Notre synthèse se concentre sur trois axes principaux. Le premier est la modélisation des jeux de fantasy cyclisme comme problèmes de sac à dos dynamiques. Le deuxième axe s'intéresse aux outils d'analyse de tolérance des problèmes de sac à dos dynamique. Tandis que le troisième présente les extensions multi-étapes du problème de sac à dos.

2.1 Modélisation des jeux de fantasy cyclisme

Le travail fondateur de l'analyse des jeux de fantasy cyclisme est dû à Beliën, Goossens et Van Reeth. Leur article de 2011 [Beliën et al., 2011] décompose le problème en trois difficultés. Tout d'abord, la résolution d'un sac à dos statique par période. Ensuite, l'introduction d'un budget dynamique par période. Enfin la limitation du nombre de transferts.

Le modèle générique de 2017 [Beliën et al., 2017] représente l'état de l'art. Il formalise le jeu comme un problème de programmation linéaire en nombres entiers (PLNE) multi-périodes dans lequel on peut acheter et vendre des joueurs à chaque période de transfert. La fonction objectif maximise la somme des points obtenus sur tous les événements, pénalise les transferts et utilise un critère secondaire de départage basé sur le budget restant. Les contraintes incluent la composition de l'équipe (nombre minimum et maximum de joueurs par type), la contrainte budgétaire dynamique, et la limitation du nombre de transferts par période. Ce cadre a inspiré notre propre modélisation, même si nous avons dû l'adapter aux règles spécifiques d'OCNC.

La revue de Durán (2021) [Durán, 2021] étudie l'utilité de ces travaux dans le cadre de l'analyse sportive. Sa revue confirme l'intérêt d'utiliser des méthodes d'optimisation dans la planification sportive.

2.2 Analyse de robustesse et de tolérance

Une fois une solution optimale obtenue pour un jeu de données fixé, une question naturelle est celle de sa *robustesse*. L'analyse de tolérance pour le problème du sac à dos, étudiée notamment par Pisinger et Saidi (2017) [Pisinger & Saidi, 2017], fournit un cadre formel. Elle calcule, pour chaque objet (coureur), l'intervalle dans lequel son profit (points) peut varier sans que l'ensemble optimal d'objets sélectionnés ne change. Appliquée à notre contexte, cette analyse permet de distinguer les coureurs "piliers", dont la sélection résiste à de fortes variations de performance, des coureurs "pivots", dont la présence dépend d'une estimation précise des points. Cette méthodologie a directement inspiré notre analyse de sensibilité présentée dans la section 3.4.1.

Plus récemment, Kumabe et Yoshida (2024) [Kumabe & Yoshida, 2024] ont introduit la notion de *sensibilité moyenne* pour le sac à dos, cherchant à concevoir des algorithmes produisant naturellement des solutions robustes.

2.3 Extensions multi-étapes du sac à dos

La dimension temporelle de notre problème le rapproche du *multistage knapsack* étudié par Bampis, Escoffier et Teiller (2022) [Bampis et al., 2022]. Dans cette variante, un ensemble d'objets évolue au cours du temps et des décisions doivent être prises séquentiellement avec des contraintes de transition. L'article montre que l'ajout de périodes et de contraintes de transition transforme radicalement la complexité algorithmique du problème, le rendant bien plus difficile que le sac à dos classique.

2.4 Positionnement de notre travail

Notre contribution peut se résumer en trois points. Premièrement, nous adaptons le modèle générique de Beliën et al. [Beliën et al., 2017] au règlement spécifique d'OCNC, en intégrant en particulier la majoration des coûts de transfert et la règle d'abandon. Deuxièmement, nous construisons une interface logicielle (Streamlit) qui rend ce modèle opérationnel

pour tout utilisateur, couplée à un module de simulation Monte Carlo qui explore la stabilité de l'équipe optimale. Troisièmement, nous proposons une analyse de sensibilité, inspirée de l'analyse de tolérance de Pisinger et Saidi, qui quantifie, pour chaque coureur et chaque période, la variation de ses performances qui provoquerait son entrée ou sa sortie de l'équipe optimale.

3 Contributions

Cette section constitue le cœur du rapport. Nous y présentons d'abord le règlement du jeu OCNC tel que nous l'avons modélisé, puis le modèle mathématique PLNE, et enfin les résultats de l'optimisation suivis des analyses post-optimales.

3.1 Présentation des règles du jeu

Le jeu "On connaît nos classiques, le jeu" (OCNC) se déroule pendant le Tour de France. Nous rappelons les règles que nous avons implémentées dans le modèle.

- **Budget et effectif.** Chaque joueur dispose d'un budget initial de 140 millions d'euros pour composer une équipe d'exactly 14 coureurs.
- **Points.** Après chaque étape, les 20 premiers coureurs marquent des points selon un barème dégressif (30, 20, 18, 17, ..., 1 points). Un bonus de 10 points est attribué au coureur désigné " Super-Combatif " à l'issue du Tour.
- **Transferts.** Aucun transfert libre n'est autorisé pendant la course. Cependant, lors des deux journées de repos (après l'étape 10 et après l'étape 15), le joueur peut remplacer les coureurs qui ont abandonné ou été disqualifiés. Les entrants sont alors facturés 10 % plus cher que leur prix initial, tandis que les sortants sont revendus à leur prix d'achat initial.
- **Classement.** Le classement final est obtenu en sommant les points marqués à chaque étape par les coureurs présents dans l'équipe au moment de l'étape. Les bonus finaux peuvent aussi s'ajouter au total pour améliorer le score.

Ces règles déterminent une structure avec trois périodes de décision :

P1 : Période 0 (étapes 1 à 10) : composition initiale.

P2 : Période 10 (étapes 11 à 15) : première fenêtre de transfert, uniquement pour remplacer des coureurs ayant abandonnés durant les 10 premières étapes.

P3 : Période 15 (étapes 16 à 21) : seconde fenêtre de transfert, pour les abandons supplémentaires survenus entre l'étape 10 et l'étape 15.

Les bonus finaux sont attribués à l'issue de l'étape 21 et sont comptabilisés pour l'équipe de la période 15.

3.2 Modèle mathématique

Nous décrivons ci-dessous la formalisation en programme linéaire en nombres entiers (PLNE) que nous avons élaborée. Elle s'inspire du cadre de Beliën et al. [Beliën et al., 2017] et en conserve la structure multi-période, tout en intégrant les spécificités d'OCNC.

Notations générales

- $\mathcal{R}$: ensemble des coureurs.
- $\mathcal{T} = \{0, 1, \dots, T_{\max}\}$: ensemble des périodes (dans OCNC, $T_{\max} = 2$, correspondant aux périodes 0, 10 et 15).
- $x_{r,t} \in \{0, 1\}$: variable binaire valant 1 si le coureur r est dans l'équipe à la période t , 0 sinon.
- $\text{buy}_{r,t} \in \{0, 1\}$: vaut 1 si le coureur r est acheté (entre dans l'équipe) à la période t ($t \geq 1$), 0 sinon.
- $\text{sell}_{r,t} \in \{0, 1\}$: vaut 1 si le coureur r est vendu (sort de l'équipe) à la période t ($t \geq 1$), 0 sinon.
- $B_t \in \mathbb{Z}_+$: budget restant à la fin de la période t .
- p_r : prix d'achat initial du coureur r .
- $\alpha \in [0, 1]$: taux de majoration appliqué aux achats en cours de compétition (pour OCNC, $\alpha = 0.1$).
- $p'_r = \lceil (1 + \alpha)p_r \rceil$: prix majoré pour les achats effectués à partir de la période $t = 1$.
- $\text{points}_{r,e}$: points obtenus par le coureur r à l'étape $e \in \{1, \dots, E\}$. Les E étapes sont réparties en $|\mathcal{T}|$ périodes selon une fonction $\text{period}(e) \in \mathcal{T}$.
- E_{final} : étape fictive pour les bonus finaux, affectée à la dernière période $T_{\max}$.
- $\mathcal{A}_t$: ensemble des coureurs ayant abandonné avant la période t . On a $\mathcal{A}_t \subseteq \mathcal{A}_{t+1}$.

Fonction objectif

Maximiser le nombre total de points :

$$\max \sum_{r \in \mathcal{R}} \sum_{e=1}^E \text{points}_{r,e} x_{r,\text{period}(e)} + \sum_{r \in \mathcal{R}} \text{points}_{r,E_{\text{final}}} x_{r,T_{\max}}.$$

Contraintes

Taille de l'équipe Pour chaque période $t \in \mathcal{T}$:

$$\sum_{r \in \mathcal{R}} x_{r,t} = N_{\text{eq}},$$

où N_{eq} est la taille constante de l'équipe (14 dans OCNC).

Dynamique des transferts Pour chaque coureur $r \in \mathcal{R}$ et pour chaque période $t \in \mathcal{T} \setminus \{0\}$:

$$\text{buy}_{r,t} - \text{sell}_{r,t} = x_{r,t} - x_{r,t-1}.$$

De plus, on interdit l'achat et la vente simultanés du même coureur :

$$\text{buy}_{r,t} + \text{sell}_{r,t} \leq 1.$$

Contraintes budgétaires Budget initial :

$$B_0 = B_{\text{initial}} - \sum_{r \in \mathcal{R}} p_r x_{r,0},$$

où B_{initial} est le budget de départ (140 M€ pour OCNC).

Évolution du budget pour $t \in \mathcal{T} \setminus \{0\}$:

$$B_t = B_{t-1} - \sum_{r \in \mathcal{R}} p'_r \text{buy}_{r,t} + \sum_{r \in \mathcal{R}} p_r \text{sell}_{r,t}, \quad B_t \geq 0.$$

Règles de transfert liées aux abandons

— Seuls les coureurs ayant abandonné peuvent être vendus à la période t :

$$\text{sell}_{r,t} = 0 \quad \forall r \notin \mathcal{A}_t, \quad \forall t \in \mathcal{T} \setminus \{0\}.$$

— Les coureurs qui ont abandonné avant la période t ne peuvent plus être présents dans l'équipe à cette période ni aux suivantes :

$$x_{r,s} = 0 \quad \forall s \geq t, \quad \forall r \in \mathcal{A}_t.$$

Domaine des variables

$$x_{r,t} \in \{0, 1\}, \quad \text{buy}_{r,t} \in \{0, 1\}, \quad \text{sell}_{r,t} \in \{0, 1\}, \quad B_t \in \mathbb{Z}_+.$$

Instanciation pour OCNC

Dans le cas concret du jeu OCNC, les paramètres prennent les valeurs suivantes :

$$B_{\text{initial}} = 140, \quad N_{\text{eq}} = 14, \quad \alpha = 0.1, \quad T_{\text{max}} = 2.$$

Les trois périodes correspondent à $\mathcal{T} = \{0, 1, 2\}$, avec la convention que la période $t = 0$ couvre les étapes 1 à 10, $t = 1$ les étapes 11 à 15, et $t = 2$ les étapes 16 à 21. Les abandons sont fournis sous la forme de deux ensembles $\mathcal{A}_1$ et $\mathcal{A}_2$ issus des données officielles de la course.

3.2.1 Implémentation et résolution

Le modèle est implémenté en Python avec la bibliothèque PuLP [S. Mitchell et al., 2011]. Le solveur utilisé est CBC (Coin-or Branch and Cut), appelé via `pulp.PULP_CBC_CMD`. La gestion des données (prix, points, abandons) est confiée au module `trait_donnees.py`, qui lit les fichiers CSV et construit les dictionnaires nécessaires. Au sein de l'application Streamlit, le modèle est construit dynamiquement dans la fonction `optimiser_equipe_pulp`, qui reçoit en paramètres le budget, la taille d'équipe, les listes d'abandons et un booléen autorisant ou non les transferts.

3.3 Présentation de la solution optimale

3.3.1 Données utilisées

Pour valider notre modèle, nous l'avons appliqué aux données du Tour de France 2025 regroupées dans le dossier `tdf_rankings`. Les coûts des coureurs sont lus depuis `valeurs.csv`. Les points d'étape sont calculés à partir des fichiers `stage_XX_2025.csv`, selon un barème appliqué aux vingt premiers de chaque étape. Les bonus liés aux maillots et à la combativité sont ajoutés depuis `maillots.csv`, tandis que les bonus du classement final sont intégrés depuis `final.csv` sous la forme d'une étape finale fictive (étape 22). Le script `trait_donnees.py` centralise la construction des dictionnaires de coûts et de points, qui sont ensuite exploités par le modèle d'optimisation. La répartition des points par période ainsi que la prise en compte des abandons sont gérées dans le modèle à travers les périodes de transfert définies dans `app_pulp.py`.

3.3.2 Équipe optimale

Avec les paramètres de référence, c'est-à-dire un budget initial de 140, une équipe de 14 coureurs et des transferts autorisés uniquement lorsqu'un coureur abandonne, le modèle trouve une solution optimale de 2 505 points. Ce total correspond aux points marqués sur les 21 étapes, auxquels s'ajoutent les bonus du classement final. Dans cette solution, le bonus final représente 562 points.

L'optimisation utilise pleinement les trois périodes définies dans le modèle. L'équipe initiale est sélectionnée pour maximiser les points des dix premières étapes tout en respectant exactement le budget disponible. Après la première vague d'abandons, le modèle remplace Wærrenskjold Søren par Abrahamsen Jonas. Après la seconde vague, Poel Mathieu est remplacé par Arensman Thymen. Ces transferts permettent de conserver une équipe complète tout en orientant la sélection vers les coureurs encore capables de marquer sur les dernières étapes et dans les classements finaux.

Le budget restant est nul au départ, vaut 1 après la première période de transfert, puis redevient nul après le second transfert. Le modèle exploite donc presque entièrement le budget disponible, ce qui montre que la contrainte budgétaire est active dans la solution optimale.

L'équipe retenue en période finale est composée de coureurs combinant de bons scores sur les dernières étapes et des bonus finaux importants. Le tableau 1 présente la composition obtenue pour la période 15, c'est-à-dire l'équipe utilisée pour les étapes 16 à 21 et pour les bonus de classement final.

TABLE 1 – Exemple d'équipe optimale en période 15 (extrait)			
Coureur	Prix (M€)	Points période 15+	Bonusfinal
Pogacar Tadej	20		259
Vingegaard Jonas	19		130
...	...		...

L'application offre également un onglet "Points et scores" où l'utilisateur peut consulter le détail des points par coureur et par étape.

3.4 Analyse post-optimale

Au-delà de la solution ponctuelle présentée, l'intérêt du projet est l'analyse post optimale. Nous avons implémenté trois outils principaux, qui sont accessibles depuis l'interface.

3.4.1 L'Analyse de sensibilité des points

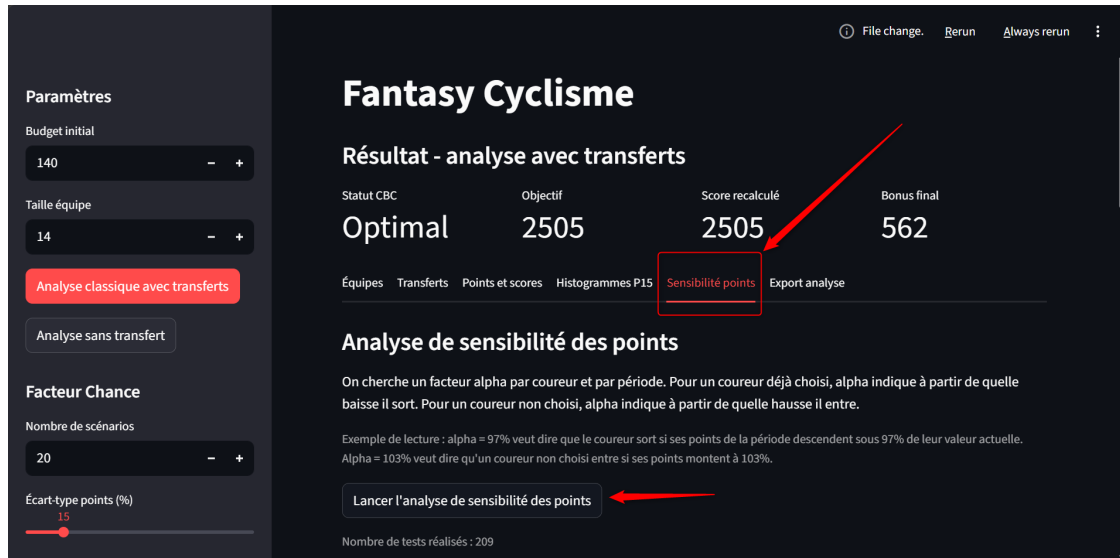


FIGURE 1 – Analyse de sensibilité

Cette première analyse, s'inspire de l'analyse de tolérance de Pisinger et Saidi [Pisinger & Saidi, 2017]. Pour chaque coureur et chaque période, nous calculons un coefficient multiplicateur α des points de cette période, qui est défini comme suit :

- *Coureur déjà dans l'équipe optimale de base.*
On résout le modèle en *interdisant* ce coureur dans sa période correspondante. Soit Δ la baisse de score observée. Alpha est calculé $\alpha = 1 - \Delta/P$, P est le total des points apportés par ce coureur sur la période analysée, bonus final inclus pour la période 15.
- Si $\alpha \leq 0$, le coureur reste optimal même avec un nombre de points nul (l'interdire fait perdre au moins autant de points que ce qu'il rapportait sur la période analysée.). NB : Dans l'interface l'affichage est tronqué à 0.0%.
- Si $\alpha \approx 1$, il suffit d'une légère baisse de ses points pour qu'il sorte de l'équipe.
- *Coureur non choisi.*
On le *force* dans l'équipe. Si le score baisse de Δ points, alors $\alpha = 1 + \Delta/P$.
- Si α est proche de 1, le coureur est presque optimal et pourrait entrer avec une très légère hausse de performance.

Le calcul est lancé pour les trois périodes et pour l'ensemble des coureurs uniquement jusqu'au dernier sélectionné dans l'équipe optimale de base, c'est-à-dire ceux qui sont classés dans l'ordre décroissant des points jusqu'à inclure le dernier coureur effectivement retenu par le modèle.

L'analyse de sensibilité met en évidence des coureurs à la fois très performants et très fragiles : un coureur qui coûte 25 millions d'euros peut, par exemple, être retiré si ses points baissent de seulement 5 %. Elle révèle aussi des coureurs sous-évalués par le modèle de base,

qui entreraient dans l'équipe avec une hausse de seulement 2 à 3 % de leurs points de la période correspondante.

Coueurs déjà dans l'équipe

Période analysée	Coureur	Points de la période	Points qu'il peut perdre	Alpha critique	Variation nécessaire	Interprétation
Période 15 : étapes 16 à 21 + bonus final	Arensman Thymen	75	2	97.3%	-2.7%	Sort sous 97.3%

FIGURE 2 – Arensman coûte 13 millions, sort si ses points baissent de 2.7%

Coueurs non choisis jusqu'au dernier choisi

Période analysée	Rang	Coureur	Points de la période	Points à gagner	Alpha critique	Variation nécessaire	Interprétation
Période 15 : étapes 16 à 21 + bonus final	8	Paret- Peintre Valentin	73	2	102.7%	+2.7%	Entre à 102.7%

FIGURE 3 – Paret-Peintre Valentin, il entre dans l'équipe avec une hausse de 2.7%

3.4.2 Couches de Pareto

Nous avons également implémenté une visualisation Couches de Pareto uniquement pour la dernière période (P15). Selon les deux critères (coût d’achat, points comptés), les coueurs sont classés en *couches de Pareto* : la couche 1 regroupe les coueurs non dominés (aucun autre coueur n’est à la fois moins cher et plus performant), la couche 2 est le front Pareto des coueurs restants après suppression de la première couche, et ainsi de suite.

Cette classification permet de repérer visuellement des indices de substitution ou d’ef-fets de contraintes. Un coueur situé en couche 2 ou 3 mais sélectionné par le modèle n’est pas optimal selon la seule lecture coût/points, sa présence peut alors s’expliquer par les contraintes globales du problème, comme le budget, les abandons ou la nécessité de com-pléter exactement l’équipe. À l’inverse, un coueur de couche 1 non sélectionné montre que son bon compromis individuel coût/points ne suffit pas nécessairement à l’intégrer dans la combinaison optimale, notamment lorsque d’autres choix consomment déjà le budget disponible.

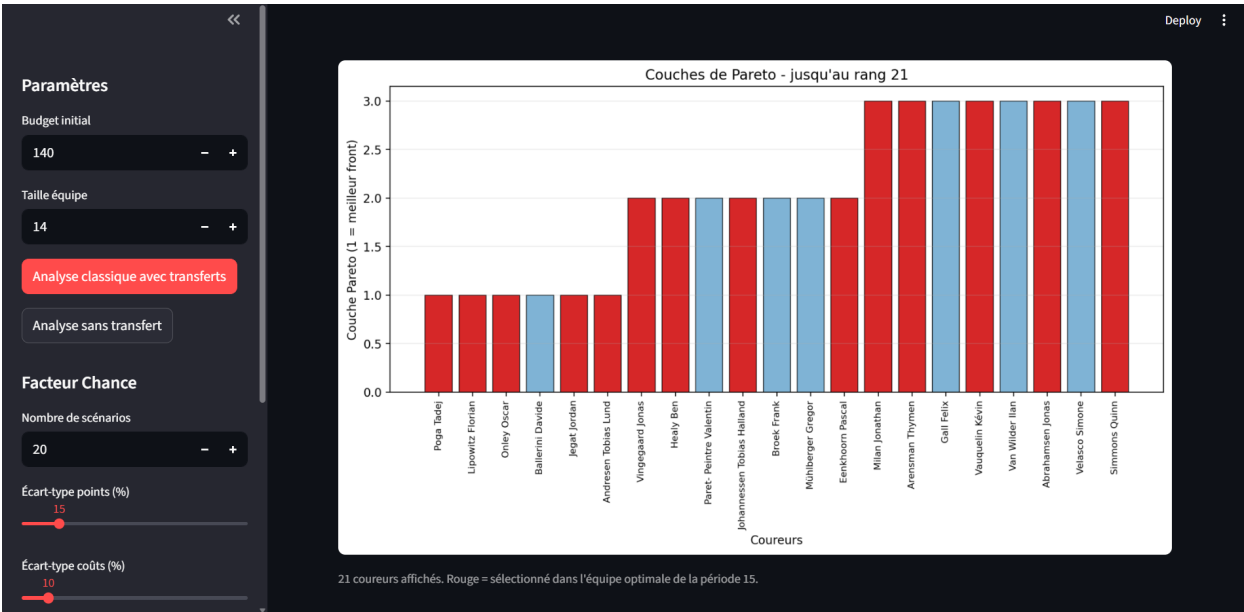


FIGURE 4 – Les couches de Pareto.

3.4.3 Stabilité par simulation Monte Carlo - Facteur Chance

Cette analyse répond à la question : *Quand les données sont perturbées, quels coureurs reviennent le plus souvent dans l'équipe optimale ? Ou bien si nous mettons en place un facteur chance à quelle fréquence un coureur sera sélectionné ?* Une approche naturelle consiste à perturber aléatoirement les points, les prix et la taille de l'équipe, puis à résoudre le modèle pour chaque scénario et compter les sélections.

Dans l'interface, nous pouvons générer jusqu'à 1500 scénarios. Pour chaque scénario :

- Les points de chaque coureur à chaque étape sont multipliés par un facteur aléatoire tiré d'une loi normale de moyenne 1 (les points de base) et d'écart-type σ_{points} réglable, par défaut il est à 15 %, $\mathcal{N}(1, \sigma_{points})$.
- Les prix initiaux sont également multipliés par un facteur gaussien d'écart-type σ_{couts} , par défaut il est à 10 %, $\mathcal{N}(1, \sigma_{couts})$.
- La taille d'équipe est tirée d'une normale $\mathcal{N}(14, \sigma_{taille})$ puis bornée entre 13 et 17, et par défaut le sigma est à 1.

Pour chaque scénario résolu, nous enregistrons quels coureurs sont sélectionnés à chaque période.

Le résultat est présenté sous forme de tableau et d'histogrammes. Les coureurs sont classés par fréquence de sélection totale, puis par fréquence en période 15.

- Un code couleur identifie ceux qui figuraient déjà dans l'équipe optimale de base.

Cette analyse met en évidence un *noyau dur* de coureurs très fréquemment sélectionnés dans les scénarios testés. Ces coureurs présentent généralement un rapport points/prix suffisamment favorable pour que les perturbations modérées appliquées aux données ne remettent pas en cause leur présence dans l'équipe optimale. À l'inverse, certains coureurs apparaissent plus rarement : ils ne deviennent optimaux que lorsque les perturbations leur sont favorables, ou lorsqu'elles défavorisent leurs concurrents directs. C'est précisément ce phénomène que nous appelons ici le *facteur chance*.

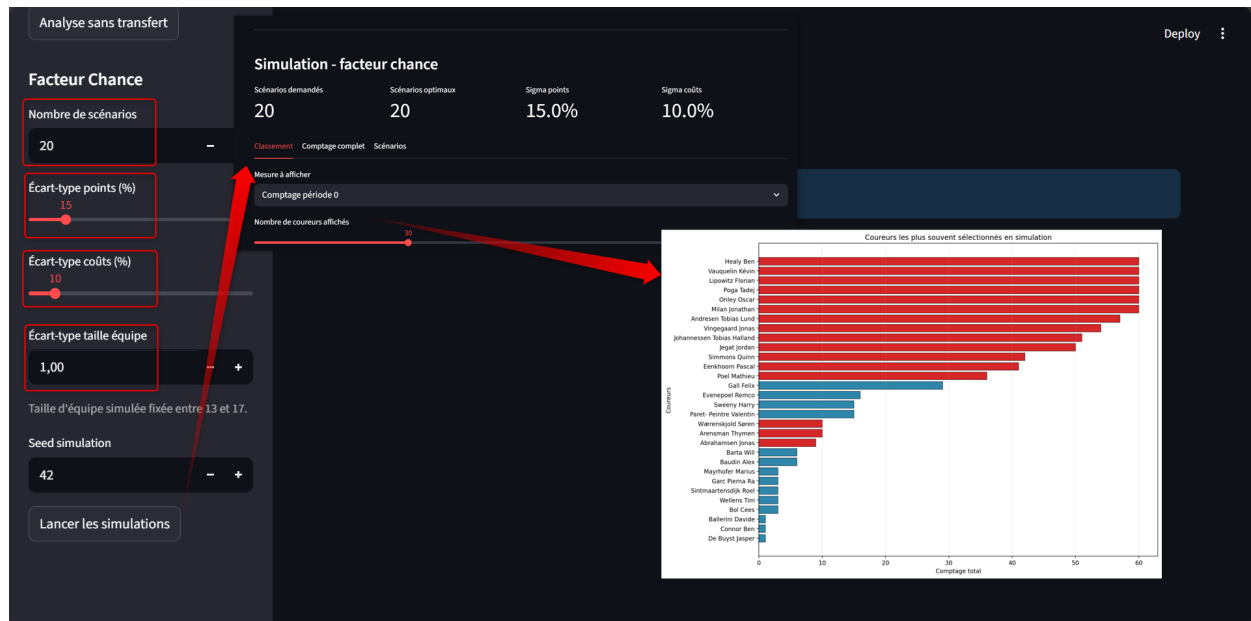


FIGURE 5 – Exemple d'un lancement d'une simulation de 20 scénarios avec $\sigma_{points} = 0.15$, $\sigma_{couts} = 0.1$ et $\sigma_{taille} = 1$

3.4.4 Analyse d'un coureur forcé

En complément, l'outil permet d'analyser un coureur particulier en l'imposant dans l'équipe de la dernière période (P15). Le modèle est alors résolu une seconde fois avec cette contrainte supplémentaire, puis le score obtenu est comparé au score optimal de base.

- Si le score reste inchangé, il existe une solution optimale alternative incluant ce coureur.
- Dans le cas contraire, l'écart de points mesure la perte de score liée à l'inclusion forcée de ce coureur.
- l'interface indique quel coureur de l'équipe optimale initiale ne figure plus dans la solution optimale contrainte.



FIGURE 6 – Exemple de l'inclusion de Paret-Peintre Valentin

4 Conclusion

Ce projet avait pour objectif de transformer le jeu de fantasy cyclisme OCNC, en un objet d'étude mathématique. Nous avons conçu et implémenté un modèle de programmation linéaire en nombres entiers qui capture l'ensemble des règles du jeu : budget, effectif, abandons, transferts. Ce modèle, résolu avec le solveur PuLP, permet de calculer *ex-post* l'équipe qui aurait maximisé les points sur le Tour de France 2025. De plus, nous avons développé une application Streamlit interactive, rendant l'outil accessible même à des utilisateurs non spécialistes.

Points forts. Au-delà du simple calcul d'une meilleure équipe, l'intérêt principal du projet réside dans les outils d'analyse post-optimale développés. L'analyse de sensibilité permet d'identifier les coureurs vraiment robustes et ceux qui sont proches de sortir ou d'entrer dans la solution. Les couches de Pareto donnent une lecture visuelle du compromis entre coût et performance. Enfin, les simulations Monte Carlo introduisent un facteur chance et montrent quels coureurs restent intéressants lorsque les points, les prix ou la taille d'équipe varient.

Points faibles et limites. Plusieurs limites doivent être soulignées. La simulation Monte Carlo, bien que très utile, repose sur des hypothèses de perturbations gaussiennes indépendantes qui ne reflètent qu'imparfaitement les incertitudes réelles (par exemple, une chute collective affecte plusieurs coureurs simultanément). De plus, la limite de 3 coureurs par maillot n'a pas été codée dans la version actuelle du modèle, car elle n'était pas présente dans le jeu de données de test. Son ajout nécessite simplement d'ajouter une contrainte de cardinalité sur le nombre de joueurs. Aussi nous n'avons pas implémenté le joker qui permet de doubler les points d'un coureur sur une étape donnée. Son ajout nécessiterait de créer pour chaque coureur une variable binaire à chaque étape. Cette implémentation augmenterait considérablement le nombre de variables, ainsi que le temps de résolution de l'algorithme.

Perspectives. Ce travail ouvre plusieurs perspectives pour approfondir notre travail. Il serait intéressant d'intégrer une analyse de dépendance entre coureurs. Cette implémentation nous permettrait de repérer la synergie entre plusieurs coureurs. À plus long terme, nous pourrions créer un modèle de machine learning nous permettant de prédire les points obtenus par les coureurs en avance. L'entraînement d'un tel modèle nécessiterait de récolter les données des anciennes éditions du Tour de France.

A Annexe A : Cahier des charges

Rappel du cahier des charges

Objectifs du projet

Développer une interface d'analyse et d'aide à la décision pour les jeux de fantasy cyclisme, en particulier pour le jeu " On connaît nos classiques " organisé par la RTBF. Les objectifs principaux comprenaient :

- **Analyse post-optimale** : À partir des données réelles d'une course (points par étape, maillots, abandons, etc.), calculer l'équipe optimale *ex post* pour chaque étape, en prenant en compte les possibilités de transferts.
- **Étude de la stabilité** : Déterminer quels coureurs sont systématiquement présents dans l'équipe optimale, et analyser l'évolution de la composition au fil des étapes.

Besoins fonctionnels

- Lecture automatique des fichiers CSV contenant les prix des coureurs (`valeurs.csv`), les résultats des étapes (`stage_XX_2025.csv`), les attributions de maillots par étape (`maillots.csv`) et les bonus finaux (`final.csv`).
- Modélisation PLNE intégrant budget de 140 M€, limite de 14 coureurs, transferts avec majoration de 10 %, restriction aux coureurs abandonnés.
- Interface utilisateur en ligne de commande ou graphique simple.
- Visualisation des équipes optimales, du score par étape et de l'analyse de stabilité.

Contraintes techniques

- Langage Python 3, solveur PuLP (CBC).
- Dépôt Git : https://github.com/Kapiloten/PAI2D_Fantasy_Cyclisme.
- Temps de résolution visé : quelques minutes pour le problème complet sur une machine standard.
- Bibliothèques supplémentaires : pandas pour la manipulation de données, éventuellement matplotlib pour les graphiques.
- Streamlit sera utilisé afin de présenter notre code sous forme d'interface graphique.

Architecture logicielle

Le code sera organisé en 2 modules :

- `trait_donnees.py` : chargement et prétraitement des données.
- `app_pulp.py` : lancement de l'interface graphique qui permettra la résolution du problème, l'analyse des résultats, ainsi que la visualisation des performances.

Organisation

- **23 janvier 2026** : Première réunion, présentation des règles et des objectifs. D'ici la prochaine réunion, réalisation de la modélisation sans transfert et obtention de la solution optimale.

- **12 février 2026** : Deuxième réunion, présentation du modèle sans transfert et de sa solution optimale, puis discussion sur la modélisation avec transferts. Pour la prochaine réunion, nous devons tester l'efficacité de Pulp sur un ensemble de test en générant aléatoirement des données (prix et scores pour un nombre variable de coureurs, ainsi que plusieurs périodes). Cela permettra d'évaluer à partir de quelle taille de problème le solveur devient trop lent.
- **24 février 2026** : Troisième réunion, présentation des résultats des tests de performance de Pulp. Présentation du modèle avec transferts que nous avons défini (en tenant compte des abandons). Tests sur les données de 2025.
- **Mars 2026** : Amélioration et finalisation du code. À l'issue de ce mois, nous disposerons d'un modèle complet incluant les transferts prêt pour les analyses.
- **Avril 2026** : Analyses post-optimales (stabilité). Nous exploiterons le modèle finalisé pour réaliser une série d'analyses approfondies sur les données : Calcul de la composition optimale pour chaque période, identification des coureurs "piliers" (présents dans un grand nombre de solution optimale avec problème perturbé)
- **Mi-avril 2026** : Quatrième réunion, validation des analyses. Nous présenterons les résultats des analyses post-optimales de stabilité. Cette réunion permettra de :
 - Valider la pertinence des observations et des méthodes employées.
 - Recueillir des retours pour affiner les analyses ou orienter le travail restant.
- **Mai 2026** : Finalisation du code, rédaction du rapport final, préparation de la soutenance. Ce dernier mois sera consacré à :
 - La finalisation et le nettoyage du code source : ajout de commentaires, documentation, README.
 - La rédaction du rapport final comprendra :
 - Une introduction présentant le contexte et les objectifs.
 - Une description détaillée de la modélisation mathématique (variables, contraintes, fonction objectif).
 - L'architecture logicielle et les choix d'implémentation.
 - Les résultats des tests de performance et des analyses post-optimales (avec graphiques et tableaux).
 - Une discussion sur les limites et les perspectives.
 - Une conclusion résumant les apports du projet.
 - La préparation de la soutenance orale : synthèse des points clés, supports visuels, répétition.
- **Fin mai 2026** : Date limite de remise du projet. Nous déposerons la version finale du code et du rapport sur le dépôt Git.

B Annexe B : Manuel utilisateur

Cette annexe décrit la prise en main du code et de l'interface Streamlit.

B.1 Installation

1. Cloner le dépôt : `git clone https://github.com/Kapiloten/PAI2D_Fantasy_Cyclisme.git`
2. Installer les dépendances requises : `pip install -r requirements.txt` (le fichier `requirements.txt` inclut `streamlit`, `pandas`, `pulp` et `matplotlib`).
3. Placer les fichiers de données du jeu dans le répertoire attendu par `trait_donnees.py` (par défaut, un sous-dossier `tdf_rankings/`).

B.2 Lancement de l'application

Depuis le répertoire du projet, exécuter la commande :

```
1 streamlit run app_pulp.py
```

Un serveur local se lance et l'interface s'ouvre dans un navigateur.

B.3 Interface principale

L'interface se compose d'une barre latérale (paramètres) et d'une zone centrale (résultats).

B.3.1 Barre latérale

- **Budget initial** : défaut 140 (millions d'euros).
- **Taille équipe** : défaut 14.
- Deux boutons de lancement :
 - **Analyse classique avec transferts** : résout le modèle PLNE avec les transferts autorisés.
 - **Analyse sans transfert** : résout le problème en forçant les variables de transfert à zéro. Permet de comparer l'apport des transferts.
- **Section Facteur Chance** :
 - **Nombre de scénarios** : nombre de simulations Monte Carlo.
 - **Écart-type points (%)** : σ pour la perturbation des points (15 % par défaut).
 - **Écart-type coûts (%)** : σ pour les prix (10 % par défaut).
 - **Écart-type taille équipe** : écart-type pour varier la taille (1 par défaut).
 - **Seed simulation** : graine aléatoire pour la reproductibilité.

Le bouton **Lancer les simulations** exécute la simulation de Monte Carlo et actualise l'affichage.

B.3.2 Zone centrale – Optimisation classique

Après avoir cliqué sur **Analyse classique avec transferts**, la zone centrale affiche les onglets suivants :

- **Équipes** : pour chaque période (0, 10, 15), le tableau des coureurs sélectionnés avec leur prix initial, leur prix d'achat transfert, les points de la période, le bonus final et une colonne points comptés (qui change uniquement pour la période 15 suite aux bonus).
- **Transferts** : tableau récapitulatif des mouvements (entrants, sortants) et des flux financiers associés.
- **Points et scores** : tableau complet des points par coureur et par étape, et graphique du score de l'équipe étape par étape.
- **Histogrammes P15** : trois graphiques pour la dernière période :
 - Points / prix initial (efficacité économique).
 - Points / coût transfert (en tenant compte du surcoût de 10 %).
 - Couches de Pareto (classement en fronts de non-domination).

Chaque histogramme affiche les coureurs en rouge (sélectionnés) ou en bleu (non sélectionnés) jusqu'au dernier coureur effectivement choisi. Un tableau détaillé donne les valeurs exactes.

- **Sensibilité points** : après avoir cliqué sur **Lancer l'analyse de sensibilité**, l'onglet affiche, pour chaque période, deux tableaux :
 - Coureurs déjà dans l'équipe, avec l'alpha de baisse qui provoquerait leur sortie.
 - Coureurs non choisis (jusqu'au dernier retenu), avec l'alpha de hausse nécessaire pour les faire entrer.

Les colonnes "Alpha critique" et "Variation nécessaire" sont exprimées en pourcentage. La colonne "interprétation" est pour faciliter la lecture.

- **Export analyse** : les données complètes (coureur, période, sélection, coût, points, bonus, statut transfert) peuvent être téléchargées au format CSV.

B.3.3 Zone centrale – Simulation Monte Carlo

Lorsqu'une simulation a été lancée, une section supplémentaire apparaît sous les résultats de l'optimisation de base. Elle contient trois onglets :

- **Classement** : histogramme horizontal des coureurs les plus fréquemment sélectionnés selon une métrique choisie (fréquence(%) ou comptage, total ou par une période précise) Un code couleur distingue les coureurs déjà présents dans l'équipe optimale de base. Un menu déroulant permet d'analyser un coureur non présent dans la base pour comprendre pourquoi il apparaît néanmoins souvent en simulation.
- **Comptage complet** : tableau de tous les coureurs avec leurs fréquences de sélection absolues et relatives, exportable en CSV.
- **Scénarios** : tableau récapitulatif de chaque scénario (statut CBC, taille d'équipe, valeur de l'objectif) et décompte des statuts du solveur.

Références

- [Beliën et al., 2011] Beliën, J., Goossens, D., Van Reeth, D., & De Boeck, L. (2011). *Using mixed-integer programming to win a cycling game*. INFORMS Transactions on Education, 11(3), 93-99. DOI : 10.1287/ited.1110.0062.
- [Beliën et al., 2017] Beliën, J., Goossens, D., & Van Reeth, D. (2017). *Optimization modeling for analyzing fantasy sport games*. INFOR : Information Systems and Operational Research, 55(4), 275-294. DOI : 10.1080/03155986.2017.1279899.
- [Durán, 2021] Durán, G. (2021). *Sports scheduling and other topics in sports analytics : a survey with special reference to Latin America*. TOP, 29(1), 125-155.
- [Pisinger & Saidi, 2017] Pisinger, D., & Saidi, A. (2017). *Tolerance analysis for 0-1 knapsack problems*. European Journal of Operational Research, 258(3), 866-876. DOI : 10.1016/j.ejor.2016.10.054.
- [Kumabe & Yoshida, 2024] Kumabe, S., & Yoshida, Y. (2024). *Average sensitivity of the knapsack problem*. arXiv preprint arXiv :2405.13343.
- [Bampis et al., 2022] Bampis, E., Escoffier, B., & Teiller, A. (2022). *Multistage knapsack*. Journal of Computer and System Sciences, 126, 106-118. DOI : 10.1016/j.jcss.2022.01.002.
- [Ausloos, 2024] Ausloos, M. (2024). *Should one (be allowed to) replace the Cipollini's ?*. Annals of Operations Research, 1-19. DOI : 10.1007/s10479-024-06206-y.
- [S. Mitchell et al., 2011] S. Mitchell, M. O'Sullivan, I. Dunning. *PuLP : A Linear Programming Toolkit for Python*, 2011. <https://coin-or.github.io/pulp/>